

NewEvent Platform Solution Design, Deployment & CI/CD Pipeline

A microservices-based event registration platform, deployed to Kubernetes (k3s) on AWS with an automated blue-green CI/CD pipeline, real-time web analytics, and infrastructure observability.

Student name: Hirusha Sasanka

Student ID:

Module / Unit:

Submission date:

Repository: cloud_computing_class_work

Table of Contents

1. Introduction
2. Solution Architecture [LO1]
3. Deployment Architecture [LO1]
4. Security & Ethics Considerations — NewEvent Platform [LO1]
5. CI/CD Process [LO1][LO2][LO3][LSEPI]
6. Implemented Services & Kubernetes Artefacts [LO4]
7. Blue-Green Deployment Implementation [LO4]
8. Deployment & Pipeline Scripts [LO4]
9. Runbook — Deployment & Testing [LO4]
10. Evidence — ClickHouse & Web Analytics
11. Evidence — Visual Analytics Dashboard
12. Conclusion

1. Introduction

NewEvent is a conference/event registration platform built as a set of independently deployable microservices. Attendees browse events and programs, register for tickets through a public web frontend, and receive confirmation email; organisers get real-time visibility into web traffic and registration behaviour through a ClickHouse-backed analytics pipeline visualised in Superset, alongside infrastructure health metrics in Prometheus/Grafana.

The platform is containerised throughout and deployed to a lightweight Kubernetes distribution (k3s) running on a single AWS EC2 instance. Delivery is fully automated: a push to the main branch triggers a CI workflow (typecheck and test), which on success triggers a CD workflow that builds and pushes container images, deploys two AWS Lambda functions, and performs a blue-green cutover of each service on the cluster with a zero-downtime rollback path.

This report documents the solution's architecture, its deployment topology, the security and ethical considerations that arise from the design, the CI/CD pipeline that delivers it, the actual scripts used to do so, a runbook for deploying and testing it, and evidence of the analytics pipeline in operation.

2. Solution Architecture

[LO1]

2.1 Architecture Diagram

INSERT DIAGRAM HERE

Solution Architecture Diagram

Must clearly distinguish request flow (e.g. solid arrows) from data flow (e.g. dashed arrows), as required. See Section 2.2 and 2.3 for the flows this diagram should depict, and docs/architecture.md in the repository for the source diagram this was drawn from.

2.2 Request Flow

All external traffic enters through a single Traefik Ingress controller, which terminates TLS and routes by hostname and path:

- **app.<domain>** — path-routed to frontend-service (static site, /) and five API backends: event-service (/api/v1/event), program-service (/api/v1/program), registration-service (/api/v1/registration), auth-service (/api/v1/auth), and analytics-service (/api/v1/collect).
- **performance.<subdomain>.<domain>** — routed entirely to grafana.

- **web.<subdomain>.<domain>** — routed entirely to superset.

Within the application, `registration-service` issues its own outbound requests: it calls `event-service` synchronously to verify and update seat availability as part of handling a registration, and invokes two AWS Lambda functions asynchronously (fire-and-forget) once a registration outcome is known — one to send a confirmation email, one to notify the attendee that their requested ticket count exceeds availability. Because these Lambda invocations are asynchronous, the registration request itself never blocks on email delivery.

2.3 Data Flow

Two datastores underpin the system, each serving a distinct purpose:

- **PostgreSQL** is the system of record for events, programs and registrations, and additionally hosts Superset's own metadata database.
- **ClickHouse** is an append-only store for web analytics events, written in batches by `analytics-service` as visitors interact with the frontend (page views, section attention, video engagement, registration funnel progress, traffic source/device/browser attribution).

Superset reads from both ClickHouse (for the analytics dashboards) and Postgres (its own metadata) but never writes application data back. Prometheus scrapes `/metrics` from each core service plus `postgres-exporter` on a fixed interval; Grafana reads from Prometheus for the infrastructure dashboards. Both dashboard tools are strictly read-only consumers of data produced elsewhere in the system.

2.4 Component Summary

Component	Responsibility	Datastore(s)
frontend-service	Static site (nginx) — event listing, registration form, admin view	—
event-service	Event CRUD, seat availability	Postgres
program-service	Conference programme/session data	Postgres
registration-service	Registration validation, seat decrement, email trigger orchestration	Postgres
auth-service	Admin authentication (JWT issuance)	—
analytics-service	Buffers and flushes web analytics events	ClickHouse
registration-confirmation (Lambda)	Sends confirmation email via SES; includes a low-seats notice when applicable	—

seats-unavailable-notifier (Lambda)	Sends a rejection email when requested tickets exceed availability	—
Superset	Visual analytics dashboards over ClickHouse	ClickHouse, Postgres (metadata)
Grafana / Prometheus	Infrastructure and service health dashboards	Prometheus (self-hosted TSDB)

3. Deployment Architecture

[LO1]

3.1 Deployment Diagram

INSERT DIAGRAM HERE

Deployment Architecture Diagram

Commit → CI → CD → (Docker Hub image push / Lambda deploy / SSH blue-green cutover) → the running k3s cluster. See Section 3.2 for the narrative and docs/architecture.md for the source diagram.

3.2 Infrastructure

The entire application runs on a single AWS EC2 instance with an associated Elastic IP (chosen specifically so the instance's public DNS hostname, and therefore every DNS record pointing at it, remains stable across instance stop/start cycles). The instance runs k3s, a lightweight, single-binary Kubernetes distribution suited to small, resource-constrained hosts; k3s bundles the Traefik ingress controller by default, which is used unmodified rather than installing a separate ingress controller.

All application resources live in a single namespace, `newevent`. Within it:

- Six services (frontend-service, event-service, program-service, registration-service, auth-service, analytics-service) are deployed using a blue-green pattern (Section 7).
- Postgres and ClickHouse run as single-replica Deployments backed by PersistentVolumeClaims, with readiness/liveness probes (pg_isready for Postgres, /ping for ClickHouse) so dependents do not attempt to connect before the datastore is actually ready.
- Prometheus, Grafana, postgres-exporter and Superset run as single-replica Deployments; cAdvisor runs as a DaemonSet since it must inspect the container runtime state of the node it runs on directly, via read-only hostPath mounts.
- Superset dashboard seeding runs as a Kubernetes Job (not a Deployment), reflecting its run-once-per-deploy nature rather than a long-running process.

Three DNS CNAME records point at the instance's Elastic-IP-backed hostname, each routed by Traefik to a different backend (frontend, Grafana, Superset). TLS is provisioned automatically via **cert-manager** using the ACME HTTP-01 challenge type — chosen because the project's DNS provider only permits CNAME/MX/SPF records and not the TXT records that the alternative DNS-01 challenge would require. cert-manager watches the cluster's Ingress resource for a cert-manager.io/cluster-issuer annotation and a tls: block, requests a Let's Encrypt certificate covering all three hostnames as Subject Alternative Names, stores it as a Kubernetes Secret, and renews it automatically ahead of expiry with no manual certificate management.

4. Security & Ethics Considerations — NewEvent Platform

[LO1]

The following issues were identified as genuine considerations arising from the platform's design, rather than a generic checklist:

4.1 Personal data handling

Registration captures an attendee's full name, email address and ticket count, stored indefinitely in Postgres with no visible retention or deletion policy. Depending on jurisdiction, this constitutes personal data under regulations such as GDPR, which would require a documented lawful basis for processing, a defined retention period, and a mechanism for an individual to request erasure. None of these currently exist in the implementation and would need to be addressed before handling real attendee data in production.

4.2 Unauthenticated email-sending surface

The registration endpoint accepts an arbitrary email address and, on success or failure, triggers an email to that address with no verification that the submitter controls it. This is a genuine abuse vector: the endpoint could be used to send unsolicited email to third parties who never interacted with the platform, which is both a security concern (spam/reputation abuse of the sending domain) and an ethical one (sending correspondence to people who did not consent to receive it). A production deployment would need rate limiting per source IP/email and, ideally, an email verification step before any message is sent to a new address.

4.3 Web analytics without visible consent

The frontend's analytics instrumentation captures page views, session behaviour, device type, browser, and referrer/UTM data for every visitor and forwards it to `analytics-service` without a visible cookie/tracking consent mechanism. Under ePrivacy-style regulation this typically requires informed opt-in before non-essential tracking begins. This is flagged here as an ethical and compliance gap in the current design, not something already mitigated.

4.4 Seat-availability race condition

Seat validation and decrement in `registration-service` reads the current `seatsAvailable` value, then issues a separate update; two near-simultaneous registrations for the same event can both read a seat count that appears sufficient before either write lands, resulting in overselling. This is a data-integrity and fairness issue (attendees believing they hold a confirmed seat that does not exist) that would need an atomic decrement (e.g. a conditional `UPDATE ... WHERE seatsAvailable >= ticketcount`) or row-level locking to resolve correctly under concurrent load.

4.5 Absence of rate limiting

No rate limiting is visible on the registration or authentication endpoints. This leaves both open to brute-force credential attempts against `auth-service` and to scripted mass-registration or scraping against `registration-service/event-service`.

5. CI/CD Process

[LO1][LO2][LO3][LSEPI]

5.1 Pipeline Diagram

INSERT DIAGRAM HERE

CI/CD Pipeline Diagram

May reuse the Deployment Architecture diagram from Section 3.1, which already depicts the full commit-to-cluster pipeline.

5.2 Process Description

CI workflow (`.github/workflows/ci.yml`) runs on every push and pull request to `main`. It typechecks every TypeScript service (`tsc --noEmit`) and runs the test suites that exist (`registration-service`, `auth-service`) across a matrix, so a broken build or failing test halts the pipeline before any deployment is attempted.

CD workflow (`.github/workflows/cd.yml`) is triggered automatically once CI completes successfully on `main`, or manually via `workflow_dispatch`. It performs, in order:

1. Logs in to Docker Hub using repository secrets.
2. Writes the application `.env` file from a single encrypted GitHub secret (`APP_ENV_FILE`) — this becomes the source of truth for the Kubernetes `app-secrets` Secret on the server.
3. **Deploys every AWS Lambda function** found under `lambda/` (currently `registration-confirmation` and `seats-unavailable-notifier`) via `lambda/deploy/deploy-all.sh`, which is idempotent: it creates a function if it does not yet exist, or updates its code/configuration if it does. Adding a new `lambda/<name>/` directory to the repository is sufficient for it to be deployed automatically on the next run — no manual AWS console step is ever required.

4. Builds and pushes container images for all six services plus Superset to Docker Hub, tagged with the commit's short SHA (deliberately never `latest` — see Section 7).
5. SSHes into the EC2 instance and runs `k8s/aws/deploy-on-server.sh`, which bootstraps `k3s` on first run (idempotent thereafter), applies the namespace/secrets/ConfigMaps, applies stateful and non-blue-green services, installs `cert-manager` and its `ClusterIssuer`, applies the Ingress, and finally performs a blue-green cutover of each of the six services in turn.

5.3 Security & Ethics Challenges of this Cloud Solution

REAL INCIDENT ENCOUNTERED DURING THIS PROJECT

While debugging a failure in the confirmation-email flow, a local development file (`.env.local`) was found still holding a live AWS access key and secret in plaintext, left over from earlier manual testing. It had never been committed (correctly excluded by `.gitignore`), but its mere existence in plaintext on a local disk is exactly the kind of exposure that leads to credential leaks in practice — for example if that machine's disk, a screen-share, or an accidental `git add -A` ever exposed it. This is retained here as a first-hand example of why secrets must live only in a managed secret store (GitHub Actions secrets, in this project's case) and never in a file that a developer might plausibly copy, share, or forget about.

Secrets management. All credentials used by the pipeline (Docker Hub token, SSH private key, the application `.env` contents, AWS credentials) are stored as encrypted GitHub Actions secrets and only ever materialised as plaintext inside the ephemeral, single-use GitHub-hosted runner for the duration of one job — never committed to the repository.

SSH key handling. The `deploy` step disables strict host-key checking (`StrictHostKeyChecking=no`) to allow non-interactive automation against a server whose host key changes if the instance is ever rebuilt. This is a deliberate trade-off: it removes protection against a man-in-the-middle attack on the very first connection to a given host, accepted here for automation convenience on a low-stakes coursework deployment, but not a trade-off that should be carried into a production system without a documented host-key pinning strategy.

Supply-chain / image provenance. Several base images are pulled by a mutable tag rather than a pinned version or digest — notably `apache/superset:latest` and the `ClickHouse/Prometheus/Grafana` images use `:latest` or a major-version-only tag. A build performed today can silently pull a different image tomorrow, which is both a reproducibility problem and a supply-chain integrity concern (an upstream maintainer or registry compromise could introduce a malicious image update transparently). The project's own service images are the exception and are deliberately tagged by commit SHA, precisely to avoid this class of problem for the artefacts it directly controls (see Section 7).

IAM scope of the deploy credential. The AWS access key used by the `Lambda-deploy` step is broad enough to create IAM roles and attach policies (required so that a brand-new Lambda function's execution role can be provisioned automatically). This is wider than strict least-privilege would recommend for an automation credential; a hardened deployment would instead pre-provision

the IAM roles out-of-band and scope the CI credential to `lambda:UpdateFunctionCode/lambda:CreateFunction` only.

Blue-green rollback integrity. The zero-downtime rollback mechanism in Section 7 depends entirely on every deployed image being uniquely and immutably tagged. Were this project to switch to `latest`-style tags, the previously "safe" idle slot's image reference would silently resolve to a different image the next time that slot's pod restarted, defeating the rollback guarantee — this is why per-commit SHA tagging is treated as a hard requirement rather than a style preference.

6. Implemented Services & Kubernetes Artefacts

[LO4]

The following services and their Kubernetes manifests were implemented as part of this solution:

Directory	Kind(s)	Purpose
k8s/00-namespace.yaml	Namespace	Isolates all application resources under newevent
k8s/01-app-config.yaml	ConfigMap	Non-secret shared configuration (DB host, service URLs, thresholds)
k8s/create-secrets.sh	Script → Secret	Builds app-secrets directly from the root .env file
k8s/create-configmaps.sh	Script → ConfigMap(s)	Generates ConfigMaps from existing source files (SQL init scripts, Prometheus/Grafana config, Superset seed script) so there is one source of truth
k8s/postgres.yaml	PVC, Deployment, Service	PostgreSQL — system of record
k8s/clickhouse.yaml	PVC, Deployment, Service	ClickHouse — analytics event store
k8s/prometheus.yaml	PVC, Deployment, Service	Metrics collection
k8s/grafana.yaml	PVC, Deployment, Service	Infrastructure dashboards
k8s/postgres-exporter.yaml	Deployment, Service	Exposes Postgres internals as Prometheus metrics
k8s/cadvisor.yaml	DaemonSet, Service	Per-node container resource metrics
k8s/superset.yaml	Deployment, Service, Job	Analytics dashboards over ClickHouse, plus one-shot dashboard seeding
k8s/cert-manager-issuer.yaml	ClusterIssuer	Let's Encrypt HTTP-01 certificate issuance
k8s/aws/ingress.yaml	Ingress	Host/path routing and TLS for the production (k3s/Traefik) environment
k8s/ingress.yaml	Ingress	Equivalent routing for local development via kind/nginx

k8s/blue-green/*.yaml	Deployment (×2 per service), Service	Blue/green pairs for each of the six user-facing services
-----------------------	---	---

Application services themselves (event-service, program-service, registration-service, auth-service, analytics-service) are implemented in TypeScript on Node.js/Express, each with its own Dockerfile, connecting to Postgres or ClickHouse as appropriate. frontend-service is a static site served by nginx. The two AWS Lambda functions (lambda/registration-confirmation, lambda/seats-unavailable-notifier) are plain Node.js handlers using the AWS SDK v3 SES client, deployed independently of the Kubernetes cluster.

7. Blue-Green Deployment Implementation

[LO4]

Each of the six user-facing services is defined as *two* Kubernetes Deployments (`<service>-blue`, `<service>-green`) and a single Service, whose label selector determines which slot currently receives live traffic. Only one slot is active at any time; the other is kept at zero replicas rather than deleted, so it can be restored instantly.

A cutover, performed by `k8s/blue-green-deploy.sh`, proceeds as follows:

1. Determine the currently active slot from the Service's selector.
2. Set the new image on the *idle* slot's Deployment and scale it from 0 to 1 replica.
3. Wait for the new pod to become ready, then port-forward directly into it (bypassing the Service entirely) and poll its health endpoint.
4. **If the smoke test fails**, the idle slot is scaled back to 0 and the script exits non-zero — the previously active slot has not been touched at any point, so this failure produces zero downtime and zero risk to production traffic.
5. **If the smoke test passes**, the Service's selector is patched to point at the new slot, cutting traffic over instantly, and the previously active slot is then scaled to 0 (kept, not deleted).

`k8s/rollback.sh` reverses this same mechanism on demand: it restores whichever slot is currently idle — which still holds whatever image was running before the last cutover, since nothing touches a slot's image while it sits at zero replicas — smoke-tests it, and cuts traffic back if that test passes.

8. Deployment & Pipeline Scripts

[LO4]

The following are the actual scripts that implement the CI/CD pipeline and deployment automation described above, reproduced in full from the repository.

8.1 CI Workflow — `.github/workflows/ci.yml`

```
name: CI Workflow

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  typecheck:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        service:
          - event-service
          - program-service
          - registration-service
          - auth-service
          - analytics-service
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22

      - name: Install dependencies
        working-directory: services/${{ matrix.service }}
        run: npm install

      - name: Typecheck
        working-directory: services/${{ matrix.service }}
        run: npx tsc --noEmit

  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        service:
          - registration-service
          - auth-service
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
```

```

- name: Install dependencies
  working-directory: services/${{ matrix.service }}
  run: npm install

- name: Run tests
  working-directory: services/${{ matrix.service }}
  run: npm test

```

8.2 CD Workflow — .github/workflows/cd.yml

```

name: CD Workflow

on:
  workflow_run:
    workflows: ["CI Workflow"]
    types: [completed]
    branches: [main]
  workflow_dispatch: {}

permissions:
  contents: read

jobs:
  deploy:
    if: github.event_name == 'workflow_dispatch' || github.event.workflow_run.conclusion == 'success'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ github.event.workflow_run.head_sha || github.sha }}

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Log in to Docker Hub
        run: echo "${{ secrets.DOCKERHUB_TOKEN }}" | docker login -u "${{ secrets.DOCKERHUB_USERNAME }}" --password-stdin

      - name: Write .env (source for the K8s app-secrets Secret)
        env:
          APP_ENV_FILE: ${{ secrets.APP_ENV_FILE }}
        run: printf '%s\n' "$APP_ENV_FILE" > .env

      - uses: actions/setup-node@v4
        with:
          node-version: 22

      - name: Deploy Lambda functions
        run: |
          export AWS_ACCESS_KEY_ID=$(grep '^AWS_ACCESS_KEY_ID=' .env | cut -d= -f2-)
          export AWS_SECRET_ACCESS_KEY=$(grep '^AWS_SECRET_ACCESS_KEY=' .env | cut -d= -f2-)
          export AWS_REGION=$(grep '^AWS_REGION=' .env | cut -d= -f2-)
          export SES_FROM_EMAIL=$(grep '^SES_FROM_EMAIL=' .env | cut -d= -f2-)
          (cd lambda/deploy && npm install)
          bash lambda/deploy/deploy-all.sh

      - name: Write SSH private key

```

```

env:
  SSH_PRIVATE_KEY: ${ secrets.SSH_PRIVATE_KEY }
run: |
  mkdir -p ~/.ssh
  printf '%s\n' "$SSH_PRIVATE_KEY" > ~/.ssh/deploy_key
  chmod 600 ~/.ssh/deploy_key

- name: Deploy (build, push to Docker Hub, SSH in for blue-green cutover)
  env:
    SERVER_HOST: ${ secrets.SERVER_HOST }
    SSH_KEY: /home/runner/.ssh/deploy_key
    DOCKERHUB_USERNAME: ${ secrets.DOCKERHUB_USERNAME }
  run: |
    export TAG="$(git rev-parse --short HEAD)"
    bash k8s/aws/deploy-aws.sh

```

8.3 Build/Push Orchestration — k8s/aws/deploy-aws.sh

```

#!/bin/bash
set -e

# Builds/pushes the 5 core service images (frontend + 4 microservices),
# then SSHes into the EC2 k8s instance to run the actual deployment there
# (k8s/aws/deploy-on-server.sh) - kubectl runs locally on the server against
# 127.0.0.1:6443, so nothing beyond SSH (22) and the app's own HTTP/HTTPS
# ports need to be reachable from outside.
#
# Required env vars: SERVER_HOST (public IP/hostname), SSH_KEY (path to the
# private key file), DOCKERHUB_USERNAME (Docker Hub namespace to push to,
# already logged in via `docker login` before this runs). Image build/push
# happens here (on whatever machine runs this - a GitHub Actions runner or
# locally) since the server itself is a small 2GB instance not worth
# burdening with builds.

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/../../" && pwd)"

: "${SERVER_HOST:?SERVER_HOST env var required (EC2 public IP)}"
: "${SSH_KEY:?SSH_KEY env var required (path to private key file)}"
: "${DOCKERHUB_USERNAME:?DOCKERHUB_USERNAME env var required (Docker Hub namespace)}"

TAG="${TAG:-$(git -C "$ROOT_DIR" rev-parse --short HEAD)}"
SSH_OPTS="-o StrictHostKeyChecking=no -i $SSH_KEY"

SERVICES="
frontend-service:services/frontend-service
event-service:services/event-service
program-service:services/program-service
registration-service:services/registration-service
auth-service:services/auth-service
analytics-service:services/analytics-service
superset:superset
"

echo "== Building and pushing images (tag: ${TAG}, platform: linux/amd64) =="
for entry in $SERVICES; do
  name="${entry%*:}"
  context="${entry#*:}"
  image="docker.io/${DOCKERHUB_USERNAME}/newevent-${name}:${TAG}"

```

```

    echo "-- ${image}"
    docker buildx build --platform linux/amd64 --push -t "$image" "${ROOT_DIR}/${context}"
done

echo "== Staging .env on the server =="
# shellcheck disable=SC2086
ssh $SSH_OPTS "ubuntu@${SERVER_HOST}" "mkdir -p ~/env-staging"
# shellcheck disable=SC2086
scp $SSH_OPTS "${ROOT_DIR}/.env" "ubuntu@${SERVER_HOST}:~/env-staging/.env"

echo "== Running deployment on the server =="
# shellcheck disable=SC2086
ssh $SSH_OPTS "ubuntu@${SERVER_HOST}" "bash -s -- '${TAG}' '${DOCKERHUB_USERNAME}'" <
"${ROOT_DIR}/k8s/aws/deploy-on-server.sh"

echo "Done."

```

8.4 Server-Side Deployment — k8s/aws/deploy-on-server.sh

```

#!/bin/bash
set -e

# Runs ON the EC2 instance itself (piped over SSH, not executed locally).
# Uses k3s's own local kubectl against 127.0.0.1:6443 - no Kubernetes API
# port needs to be exposed to the internet at all with this approach, only
# SSH (22) does, which is what actually reaches this script in the first
# place.
#
# Bootstraps k3s itself on first run (idempotent - skipped once k3s is
# already installed). Relies on the default Ubuntu AMI's passwordless sudo
# for the `ubuntu` user, since this runs non-interactively over SSH.
#
# Usage (from the caller, over ssh): bash -s -- <tag> <dockerhub-username>
# Expects ~/env-staging/.env to already be in place (written by the caller
# via scp before this runs) - contains the secrets for k8s/create-secrets.sh.

TAG="$1"
DOCKERHUB_USERNAME="$2"
APP_DIR="$HOME/app"

if [ -z "$TAG" ] || [ -z "$DOCKERHUB_USERNAME" ]; then
    echo "Usage: deploy-on-server.sh <tag> <dockerhub-username>" >&2
    exit 1
fi

export PATH="$PATH:/usr/local/bin"

if ! command -v k3s >/dev/null 2>&1; then
    echo "== k3s not found - installing (first run on this server) =="
    curl -sL https://get.k3s.io | sh -
    echo "== Waiting for k3s node to be Ready =="
    until sudo k3s kubectl get nodes 2>/dev/null | grep -q " Ready"; do
        sleep 2
    done
fi

mkdir -p "$HOME/.kube"
sudo cp /etc/rancher/k3s/k3s.yaml "$HOME/.kube/config"

```

```

sudo chown "$(id -u):$(id -g)" "$HOME/.kube/config"
chmod 600 "$HOME/.kube/config"

if [ -d "$APP_DIR/.git" ]; then
    echo "== Updating existing checkout =="
    git -C "$APP_DIR" fetch --depth 1 origin main
    git -C "$APP_DIR" reset --hard origin/main
else
    echo "== Cloning repo =="
    git clone --depth 1 https://github.com/amasha2456/cloud_computing_class_work.git "$APP_DIR"
fi

cp "$HOME/env-staging/.env" "$APP_DIR/.env"
cd "$APP_DIR"

export KUBECONFIG="$HOME/.kube/config"

echo "== Applying namespace, secrets, configmaps (idempotent) =="
kubectl apply -f k8s/00-namespace.yaml
bash k8s/create-secrets.sh
bash k8s/create-configmaps.sh
kubectl apply -f k8s/01-app-config.yaml

echo "== Applying stateful services (idempotent) =="
kubectl apply -f k8s/postgres.yaml
kubectl apply -f k8s/clickhouse.yaml
kubectl apply -f k8s/prometheus.yaml
kubectl apply -f k8s/grafana.yaml
kubectl rollout status deployment/postgres -n newevent --timeout=120s
kubectl rollout status deployment/clickhouse -n newevent --timeout=120s

echo "== Applying non-blue-green app services =="
kubectl apply -f k8s/postgres-exporter.yaml
kubectl apply -f k8s/cadvisor.yaml
kubectl apply -f k8s/superset.yaml
kubectl set image deployment/superset "superset=docker.io/${DOCKERHUB_USERNAME}/newevent-superset:${TAG}" -n newevent
kubectl rollout status deployment/superset -n newevent --timeout=300s || \
    echo "WARNING: superset rollout did not finish in time, check: kubectl get pods -n newevent -l app=superset"
kubectl rollout status deployment/postgres-exporter -n newevent --timeout=60s || \
    echo "WARNING: postgres-exporter rollout did not finish in time, check: kubectl get pods -n newevent -l app=postgres-exporter"

echo "== Applying cert-manager (idempotent - installs on first run, no-op after) =="
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.16.2/cert-manager.yaml
kubectl wait --for=condition=Available deployment/cert-manager -n cert-manager --timeout=120s
kubectl wait --for=condition=Available deployment/cert-manager-webhook -n cert-manager --timeout=120s
kubectl wait --for=condition=Available deployment/cert-manager-cainjector -n cert-manager --timeout=120s
kubectl apply -f k8s/cert-manager-issuer.yaml

echo "== Applying ingress (idempotent) =="
kubectl apply -f k8s/aws/ingress.yaml

SERVICES="analytics-service event-service program-service registration-service auth-service frontend-service"
for name in $SERVICES; do
    image="docker.io/${DOCKERHUB_USERNAME}/newevent-${name}:${TAG}"

```

```

if kubectl get deployment "${name}-blue" -n neuevent >/dev/null 2>&1; then
    echo "== ${name}: already deployed - blue-green cutover to ${image} =="
    bash k8s/blue-green-deploy.sh "$name" "$image"
else
    echo "== ${name}: first deploy - bootstrapping with blue active =="
    kubectl apply -f "k8s/blue-green/${name}.yaml"
    kubectl set image "deployment/${name}-blue" "${name}=${image}" -n neuevent
    kubectl rollout status "deployment/${name}-blue" -n neuevent --timeout=180s || \
        echo "WARNING: ${name}-blue rollout did not finish in time, check: kubectl get pods -n
newevent -l app=${name}"
    fi
done

echo "== Waiting for superset-seed job =="
kubectl wait --for=condition=complete job/superset-seed -n neuevent --timeout=180s || \
    echo "WARNING: superset-seed job did not complete in time, check logs: kubectl logs
job/superset-seed -n neuevent"

echo "Done. kubectl get pods -n neuevent:"
kubectl get pods -n neuevent

```

8.5 Blue-Green Cutover — k8s/blue-green-deploy.sh

```

#!/bin/bash
set -e

# Blue-green cutover for one of the 7 user-facing services.
# Usage: blue-green-deploy.sh <service-name> <image-tag>
#
# Only needed for updates - the initial `kubectl apply` of the manifests in
# k8s/blue-green/ already starts with slot=blue active and slot=green idle
# (replicas: 0), so there's nothing to cut over to on a first deploy.

SERVICE="$1"
IMAGE="$2"
NAMESPACE="neuevent"

if [ -z "$SERVICE" ] || [ -z "$IMAGE" ]; then
    echo "Usage: $0 <service-name> <image-tag>" >&2
    exit 1
fi

HEALTH_PATH="/health"
if [ "$SERVICE" = "frontend-service" ]; then
    HEALTH_PATH="/"
fi

CONTAINER_PORT=$(kubectl get service "$SERVICE" -n "$NAMESPACE" -o
jsonpath='{.spec.ports[0].port}')
CURRENT_SLOT=$(kubectl get service "$SERVICE" -n "$NAMESPACE" -o
jsonpath='{.spec.selector.slot}')

if [ "$CURRENT_SLOT" = "blue" ]; then
    TARGET_SLOT="green"
else
    TARGET_SLOT="blue"
fi

```

```

echo "[$SERVICE] current active slot: $CURRENT_SLOT -> deploying $IMAGE to slot: $TARGET_SLOT"

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
case "$IMAGE" in
    docker.io/*)
        echo "[$SERVICE] image is a registry reference - the cluster will pull it directly, no local
load needed."
        ;;
    *)
        bash "$SCRIPT_DIR/load-image.sh" "$IMAGE"
        ;;
esac

kubectl set image "deployment/${SERVICE}-${TARGET_SLOT}" "${SERVICE}=${IMAGE}" -n "$NAMESPACE"
kubectl scale "deployment/${SERVICE}-${TARGET_SLOT}" --replicas=1 -n "$NAMESPACE"
kubectl rollout status "deployment/${SERVICE}-${TARGET_SLOT}" -n "$NAMESPACE" --timeout=120s

echo "[$SERVICE] smoke-testing $TARGET_SLOT slot..."
LOCAL_PORT=$((10000 + RANDOM % 10000))
kubectl port-forward "deployment/${SERVICE}-${TARGET_SLOT}" "${LOCAL_PORT}:${CONTAINER_PORT}" -n
"$NAMESPACE" >/tmp/pf-${SERVICE}.log 2>&1 &
PF_PID=$!

# Port-forward tunnel setup time varies (near-instant for local kind,
# a couple seconds for a remote cluster over the internet) - retry instead
# of a single fixed-delay attempt.
HTTP_CODE="000"
for attempt in $(seq 1 10); do
    sleep 1
    HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}")
    "http://localhost:${LOCAL_PORT}${HEALTH_PATH}" || echo "000"
    if [ "$HTTP_CODE" = "200" ]; then
        break
    fi
done
kill "$PF_PID" 2>/dev/null || true
wait "$PF_PID" 2>/dev/null || true

if [ "$HTTP_CODE" != "200" ]; then
    echo "[$SERVICE] smoke test FAILED (HTTP $HTTP_CODE) - not cutting over. Rolling back image on
$TARGET_SLOT." >&2
    kubectl scale "deployment/${SERVICE}-${TARGET_SLOT}" --replicas=0 -n "$NAMESPACE"
    exit 1
fi

echo "[$SERVICE] smoke test passed (HTTP $HTTP_CODE) - cutting traffic over to $TARGET_SLOT"
kubectl patch service "$SERVICE" -n "$NAMESPACE" -p '{"spec":{"selector":{"app":"${SERVICE}","slot":"${TARGET_SLOT}"}}}'

echo "[$SERVICE] waiting for endpoints/ingress to converge on the new slot before draining the
old one..."
sleep 8

echo "[$SERVICE] scaling down old slot: $CURRENT_SLOT (kept at 0 replicas for instant rollback)"
kubectl scale "deployment/${SERVICE}-${CURRENT_SLOT}" --replicas=0 -n "$NAMESPACE"

echo "[$SERVICE] done. Active slot is now $TARGET_SLOT running $IMAGE."

```

8.6 Blue-Green Rollback — k8s/rollback.sh

```
#!/bin/bash
set -e

# Rolls a service back to whichever blue-green slot is currently idle,
# reversing blue-green-deploy.sh's cutover. The idle slot was scaled to 0
# but never had its image changed, so it still runs whatever was live
# before the last cutover – scaling it back up restores exactly that.
#
# Usage: rollback.sh <service-name>

SERVICE="$1"
NAMESPACE="newevent"

if [ -z "$SERVICE" ]; then
    echo "Usage: $0 <service-name>" >&2
    exit 1
fi

HEALTH_PATH="/health"
if [ "$SERVICE" = "frontend-service" ]; then
    HEALTH_PATH="/"
fi

CONTAINER_PORT=$(kubectl get service "$SERVICE" -n "$NAMESPACE" -o
jsonpath='{.spec.ports[0].port}')
CURRENT_SLOT=$(kubectl get service "$SERVICE" -n "$NAMESPACE" -o
jsonpath='{.spec.selector.slot}')

if [ "$CURRENT_SLOT" = "blue" ]; then
    TARGET_SLOT="green"
else
    TARGET_SLOT="blue"
fi

TARGET_IMAGE=$(kubectl get deployment "${SERVICE}-${TARGET_SLOT}" -n "$NAMESPACE" -o jsonpath=
{.spec.template.spec.containers[0].image})

echo "[${SERVICE}] currently active slot: $CURRENT_SLOT -> rolling back to slot: $TARGET_SLOT
($TARGET_IMAGE)"

kubectl scale "deployment/${SERVICE}-${TARGET_SLOT}" --replicas=1 -n "$NAMESPACE"
kubectl rollout status "deployment/${SERVICE}-${TARGET_SLOT}" -n "$NAMESPACE" --timeout=120s

echo "[${SERVICE}] smoke-testing $TARGET_SLOT slot..."
LOCAL_PORT=$((10000 + RANDOM % 10000))
kubectl port-forward "deployment/${SERVICE}-${TARGET_SLOT}" "${LOCAL_PORT}:${CONTAINER_PORT}" -n
"$NAMESPACE" >/tmp/pf-${SERVICE}-rollback.log 2>&1 &
PF_PID=$!

HTTP_CODE="000"
for attempt in $(seq 1 10); do
    sleep 1
    HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}")
    "http://localhost:${LOCAL_PORT}${HEALTH_PATH}" || echo "000"
    if [ "$HTTP_CODE" = "200" ]; then
        break
    fi
done
```

```

kill "$PF_PID" 2>/dev/null || true
wait "$PF_PID" 2>/dev/null || true

if [ "$HTTP_CODE" != "200" ]; then
    echo "[$SERVICE] smoke test FAILED (HTTP $HTTP_CODE) on rollback target $TARGET_SLOT - not
cutting over. Scaling it back down." >&2
    kubectl scale "deployment/${SERVICE}-${TARGET_SLOT}" --replicas=0 -n "$NAMESPACE"
    exit 1
fi

echo "[$SERVICE] smoke test passed (HTTP $HTTP_CODE) - cutting traffic back to $TARGET_SLOT"
kubectl patch service "$SERVICE" -n "$NAMESPACE" -p "{\"spec\":{\"selector\":{\"app\":\"${SERVICE}\",\"slot\":\"${TARGET_SLOT}\"}}}"

echo "[$SERVICE] waiting for endpoints/ingress to converge on the restored slot before draining
the bad one..."
sleep 8

echo "[$SERVICE] scaling down slot: $CURRENT_SLOT"
kubectl scale "deployment/${SERVICE}-${CURRENT_SLOT}" --replicas=0 -n "$NAMESPACE"

echo "[$SERVICE] rollback done. Active slot is now $TARGET_SLOT running $TARGET_IMAGE."

```

8.7 Secret Generation — k8s/create-secrets.sh

```

#!/bin/bash
set -e

# Builds the K8s "app-secrets" Secret directly from the root .env file.
# Never commit real secret values into a YAML file in this repo.
#
# Note: root .env stores ADMIN_PASSWORD_HASH with `$` escaped as `$$`
# (a docker-compose interpolation quirk - bcrypt hashes contain literal `$`
# characters that docker-compose's ${VAR} substitution misreads as nested
# variable references). kubectl's --from-env-file does no such interpolation,
# so that escaping must be undone here, or the hash breaks silently.

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
ENV_FILE="$ROOT_DIR/.env"

if [ ! -f "$ENV_FILE" ]; then
    echo "Missing $ENV_FILE" >&2
    exit 1
fi

TMP_ENV=$(mktemp)
trap 'rm -f "$TMP_ENV"' EXIT

# Copy everything except ADMIN_PASSWORD_HASH as-is, then re-add it unescaped.
grep -v '^ADMIN_PASSWORD_HASH=' "$ENV_FILE" > "$TMP_ENV"
HASH_LINE=$(grep '^ADMIN_PASSWORD_HASH=' "$ENV_FILE" | sed 's/\$/\$/g')
echo "$HASH_LINE" >> "$TMP_ENV"

kubectl create namespace neuevent --dry-run=client -o yaml | kubectl apply -f -

kubectl create secret generic app-secrets \
    --from-env-file="$TMP_ENV" \
    -n neuevent \

```

```
--dry-run=client -o yaml | kubectl apply -f -

echo "app-secrets created/updated in namespace neuevent."
```

8.8 ConfigMap Generation — k8s/create-configmaps.sh

```
#!/bin/bash
set -e

# Generates ConfigMaps directly from existing source files, so there's a
# single source of truth (no content duplicated into hand-written YAML).

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"

kubectl create configmap postgres-init \
  --from-file=init.sql="$ROOT_DIR/db/init.sql" \
  -n neuevent --dry-run=client -o yaml | kubectl apply -f -

kubectl create configmap clickhouse-init \
  --from-file=init.sql="$ROOT_DIR/clickhouse/init.sql" \
  -n neuevent --dry-run=client -o yaml | kubectl apply -f -

kubectl create configmap prometheus-config \
  --from-file=prometheus.yml="$ROOT_DIR/observability/prometheus.yml" \
  -n neuevent --dry-run=client -o yaml | kubectl apply -f -

kubectl create configmap grafana-datasource \
  --from-file="$ROOT_DIR/observability/grafana/provisioning/datasources/prometheus.yml" \
  -n neuevent --dry-run=client -o yaml | kubectl apply -f -

kubectl create configmap grafana-dashboard-provider \
  --from-file="$ROOT_DIR/observability/grafana/provisioning/dashboards/dashboards.yml" \
  -n neuevent --dry-run=client -o yaml | kubectl apply -f -

kubectl create configmap grafana-dashboards \
  --from-file="$ROOT_DIR/observability/grafana/dashboards/" \
  -n neuevent --dry-run=client -o yaml | kubectl apply -f -

kubectl create configmap superset-seed-script \
  --from-file=seed-dashboard.mjs="$ROOT_DIR/superset/seed-dashboard.mjs" \
  -n neuevent --dry-run=client -o yaml | kubectl apply -f -

echo "ConfigMaps created/updated in namespace neuevent."
```

8.9 Lambda Fleet Deployment — lambda/deploy/deploy-all.sh

```
#!/bin/bash
set -e

# Deploys every Lambda function in this repo - each subdirectory of lambda/
# (other than deploy/ itself) that has an index.mjs is treated as one
# function, named after its directory. For each one this npm installs its
# production deps, zips it, and hands it to deploy.mjs, which creates the
# function if it doesn't exist yet or updates its code/config if it does.
#
```

```

# Safe to run on every CD run regardless of what changed: adding a new
# lambda/<name>/ directory is enough for it to get deployed automatically
# next time this runs - no manual AWS step required, and no manual step
# needed when an existing function's code changes either.
#
# Required env vars: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, SES_FROM_EMAIL
# (same as deploy.mjs - AWS_REGION is optional, defaults to us-east-2).

: "${AWS_ACCESS_KEY_ID:?AWS_ACCESS_KEY_ID env var required}"
: "${AWS_SECRET_ACCESS_KEY:?AWS_SECRET_ACCESS_KEY env var required}"
: "${SES_FROM_EMAIL:?SES_FROM_EMAIL env var required}"

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"

for dir in "$ROOT_DIR"/*; do
  name="$(basename "$dir")"
  [ "$name" = "deploy" ] && continue
  [ -f "${dir}index.mjs" ] || continue

  echo "== Deploying lambda: ${name} =="
  (cd "$dir" && npm install --omit=dev --no-audit --no-fund)

  ZIP_PATH="${dir}function.zip"
  rm -f "$ZIP_PATH"
  (cd "$dir" && zip -rq function.zip . -x ".gitignore" "function.zip")

  FUNCTION_NAME="$name" ZIP_PATH="$ZIP_PATH" node "$ROOT_DIR/deploy/deploy.mjs"
  rm -f "$ZIP_PATH"
done

```

8.10 Lambda Create/Update Logic — lambda/deploy/deploy.mjs

```

import {
  IAMClient,
  CreateRoleCommand,
  GetRoleCommand,
  AttachRolePolicyCommand,
  PutRolePolicyCommand,
} from "@aws-sdk/client-iam";
import {
  LambdaClient,
  CreateFunctionCommand,
  GetFunctionCommand,
  UpdateFunctionCodeCommand,
  UpdateFunctionConfigurationCommand,
  waitUntilFunctionActiveV2,
  waitUntilFunctionUpdatedV2,
} from "@aws-sdk/client-lambda";
import { readFileSync } from "node:fs";

const REGION = process.env.AWS_REGION || "us-east-2";
const FUNCTION_NAME = process.env.FUNCTION_NAME;
const ROLE_NAME = process.env.ROLE_NAME || `${FUNCTION_NAME}-role`;
const SES_FROM_EMAIL = process.env.SES_FROM_EMAIL;
const LOW_SEATS_NOTIFY_EMAIL = process.env.LOW_SEATS_NOTIFY_EMAIL;
const ZIP_PATH = process.env.ZIP_PATH;

if (!FUNCTION_NAME || !SES_FROM_EMAIL || !ZIP_PATH) {

```

```

    console.error("FUNCTION_NAME, SES_FROM_EMAIL and ZIP_PATH env vars are required");
    process.exit(1);
}

const envVars = { SES_FROM_EMAIL };
if (LOW_SEATS_NOTIFY_EMAIL) envVars.LOW_SEATS_NOTIFY_EMAIL = LOW_SEATS_NOTIFY_EMAIL;

const iam = new IAMClient({ region: REGION });
const lambda = new LambdaClient({ region: REGION });

const trustPolicy = JSON.stringify({
  Version: "2012-10-17",
  Statement: [
    {
      Effect: "Allow",
      Principal: { Service: "lambda.amazonaws.com" },
      Action: "sts:AssumeRole",
    },
  ],
});

const sesPolicy = JSON.stringify({
  Version: "2012-10-17",
  Statement: [
    {
      Effect: "Allow",
      Action: ["ses:SendEmail", "ses:SendRawEmail"],
      Resource: "*",
    },
  ],
});

async function ensureRole() {
  try {
    const existing = await iam.send(new GetRoleCommand({ RoleName: ROLE_NAME }));
    console.log(`Role ${ROLE_NAME} already exists: ${existing.Role.Arn}`);
    return existing.Role.Arn;
  } catch (e) {
    if (e.name !== "NoSuchEntityException") throw e;
  }
}

const created = await iam.send(
  new CreateRoleCommand({
    RoleName: ROLE_NAME,
    AssumeRolePolicyDocument: trustPolicy,
    Description: `Execution role for the ${FUNCTION_NAME} Lambda`,
  }),
);
console.log(`Created role: ${created.Role.Arn}`);

await iam.send(
  new AttachRolePolicyCommand({
    RoleName: ROLE_NAME,
    PolicyArn: "arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole",
  }),
);
console.log("Attached AWSLambdaBasicExecutionRole");

await iam.send(
  new PutRolePolicyCommand({
    RoleName: ROLE_NAME,

```

```

        PolicyName: `${FUNCTION_NAME}-ses-send`,
        PolicyDocument: sesPolicy,
    }},
);
console.log("Attached inline SES send policy");

return created.Role.Arn;
}

async function ensureFunction(roleArn) {
    const zipBytes = readFileSync(ZIP_PATH);

    let exists = false;
    try {
        await lambda.send(new GetFunctionCommand({ FunctionName: FUNCTION_NAME }));
        exists = true;
    } catch (e) {
        if (e.name !== "ResourceNotFoundException") throw e;
    }

    if (exists) {
        console.log(`Function ${FUNCTION_NAME} exists, updating code + config...`);
        await lambda.send(
            new UpdateFunctionCodeCommand({
                FunctionName: FUNCTION_NAME,
                ZipFile: zipBytes,
            }),
        );
        await waitUntilFunctionUpdatedV2(
            { client: lambda, maxWaitTime: 60 },
            { FunctionName: FUNCTION_NAME },
        );
        await lambda.send(
            new UpdateFunctionConfigurationCommand({
                FunctionName: FUNCTION_NAME,
                Runtime: "nodejs22.x",
                Handler: "index.handler",
                Timeout: 10,
                MemorySize: 128,
                Environment: { Variables: envVars },
                Role: roleArn,
            }),
        );
        await waitUntilFunctionUpdatedV2(
            { client: lambda, maxWaitTime: 60 },
            { FunctionName: FUNCTION_NAME },
        );
        console.log("Updated existing function.");
        return;
    }

    console.log(`Creating function ${FUNCTION_NAME}...`);
    await lambda.send(
        new CreateFunctionCommand({
            FunctionName: FUNCTION_NAME,
            Runtime: "nodejs22.x",
            Handler: "index.handler",
            Role: roleArn,
            Code: { ZipFile: zipBytes },
            Timeout: 10,
            MemorySize: 128,

```

```

    Environment: { Variables: envVars },
    Description: `Deployed via lambda/deploy/deploy.mjs (${FUNCTION_NAME})`,
  }},
);

await waitUntilFunctionActiveV2(
  { client: lambda, maxWaitTime: 60 },
  { FunctionName: FUNCTION_NAME },
);
console.log("Function created and active.");
}

const roleArn = await ensureRole();
console.log("Waiting 10s for IAM role propagation...");
await new Promise((r) => setTimeout(r, 10000));
await ensureFunction(roleArn);
console.log("Done.");

```

8.11 TLS Certificate Issuer — k8s/cert-manager-issuer.yaml

```

apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: amashavidumini19@gmail.com
    privateKeySecretRef:
      name: letsencrypt-prod-account-key
    solvers:
      - http01:
          ingress:
            ingressClassName: traefik

```

8.12 Production Ingress — k8s/aws/ingress.yaml

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: newevent-ingress
  namespace: newevent
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: traefik
  tls:
    - hosts:
        - app.amashacloud.fredev.app
        - performance.metrics.amashacloud.fredev.app
        - web.analytics.amashacloud.fredev.app
      secretName: newevent-tls
  rules:
    - host: app.amashacloud.fredev.app
      http:

```

```
paths:
  - path: /api/v1/event
    pathType: Prefix
    backend:
      service:
        name: event-service
        port:
          number: 5001
  - path: /api/v1/program
    pathType: Prefix
    backend:
      service:
        name: program-service
        port:
          number: 5002
  - path: /api/v1/registration
    pathType: Prefix
    backend:
      service:
        name: registration-service
        port:
          number: 5003
  - path: /api/v1/auth
    pathType: Prefix
    backend:
      service:
        name: auth-service
        port:
          number: 5006
  - path: /api/v1/collect
    pathType: Prefix
    backend:
      service:
        name: analytics-service
        port:
          number: 5005
  - path: /
    pathType: Prefix
    backend:
      service:
        name: frontend-service
        port:
          number: 80
- host: performance.metrics.amashacloud.fredev.app
  http:
    paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: grafana
            port:
              number: 3000
- host: web.analytics.amashacloud.fredev.app
  http:
    paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: superset
```

port:
number: 8088

9. Runbook — Deployment & Testing

[LO4]

9.1 Prerequisites

- An AWS EC2 instance (Ubuntu) with an associated Elastic IP, security group inbound rules for TCP 22, 80 and 443 from `0.0.0.0/0`.
- A Docker Hub account and access token.
- An AWS IAM user/access key with permission to manage Lambda functions, IAM roles and SES sending.
- A domain (or subdomain) with at least CNAME record access, pointed at the EC2 instance's Elastic-IP-backed public DNS hostname.
- GitHub repository secrets configured: `DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`, `SERVER_HOST`, `SSH_PRIVATE_KEY`, `APP_ENV_FILE`.

9.2 Deploying the Platform

1. Push to (or merge into) `main`. This triggers the **CI Workflow** automatically.
2. On CI success, the **CD Workflow** fires automatically (or trigger it manually via *Actions* → *CD Workflow* → *Run workflow*).
3. CD builds and pushes all service images to Docker Hub, deploys/updates both Lambda functions, then SSHes into the EC2 instance to apply the full Kubernetes state and perform the blue-green cutover of each service.
4. On first-ever run, allow extra time: k3s installs itself, and cert-manager (a new component) installs and issues the first TLS certificate via Let's Encrypt HTTP-01, which involves a real round-trip to Let's Encrypt's servers.
5. Confirm the run completed by checking the Actions log for `Done. kubectl get pods -n neuevent:` and reviewing the pod list it prints.

9.3 Testing the Deployment

1. SSH into the instance and run `kubectl get pods -n neuevent` — every service should show one pod `Running` (its active blue or green slot) and one idle slot at zero replicas.
2. Visit the frontend hostname in a browser and confirm the event listing loads.
3. Submit a test registration with a real, accessible email address and a ticket count within the available seats. Confirm a confirmation email arrives.

4. Submit a registration requesting more tickets than are available. Confirm the API returns a rejection and a "seats unavailable" email arrives instead.
5. Visit the Grafana hostname and confirm the infrastructure/service dashboards render with live data.
6. Visit the Superset hostname and confirm the analytics dashboard renders with data captured from the frontend visits performed in the previous steps (see Section 10).
7. Optionally exercise the rollback path: run `bash k8s/rollback.sh <service-name>` on the server and confirm traffic cuts back to the previous slot without downtime.

10. Evidence — ClickHouse Deployment & Captured Web Analytics

ClickHouse is deployed via `k8s/clickhouse.yaml` with the schema below (`clickhouse/init.sql`), applied automatically as a ConfigMap-mounted init script on first startup.

```
CREATE DATABASE IF NOT EXISTS analytics;

CREATE TABLE IF NOT EXISTS analytics.web_events (
  event_id      String,
  event_type    LowCardinality(String),
  visitor_id    String,
  session_id    String,
  page_path     String,
  section_id    LowCardinality(String),
  target        String,
  label         String,
  value         Float64,
  referrer      String,
  utm_source     LowCardinality(String),
  utm_medium     LowCardinality(String),
  utm_campaign   LowCardinality(String),
  device_type    LowCardinality(String),
  browser        LowCardinality(String),
  os             LowCardinality(String),
  country        LowCardinality(String),
  properties     Map(String, String),
  client_ts      DateTime64(3, 'UTC'),
  server_ts      DateTime64(3, 'UTC') DEFAULT now64(3, 'UTC'),
  event_date     Date DEFAULT toDate(server_ts)
) ENGINE = MergeTree
PARTITION BY toYYYYMM(event_date)
ORDER BY (event_type, server_ts);
```

INSERT EVIDENCE HERE

Screenshot(s) proving ClickHouse is running and contains captured events

Suggested evidence: `kubectl get pods -n neuevent -l app=clickhouse` showing a Running pod, and a query result such as `SELECT count(), event_type FROM analytics.web_events GROUP BY event_type` run against the deployed instance, showing real captured rows.

11. Evidence — Visual Analytics Dashboard

INSERT EVIDENCE HERE

Screenshot(s) of the Superset dashboard visualising ClickHouse analytics data

The seeded dashboard is titled "New Event — Visitor & Registration Analytics" and includes charts for the Registration Funnel, Sold-Out Rejections Over Time, Section Attention Funnel, Program/Track Interest, Speaker Interest, Video Engagement, Traffic Sources, Device Mix and Browser Mix, all sourced from the `web_events` table shown in Section 10.

12. Conclusion

This report has described NewEvent's solution and deployment architecture, the security and ethical considerations that arise from its design, the CI/CD pipeline that automates its delivery end-to-end (including Lambda functions and blue-green Kubernetes cutovers), the scripts that implement that automation, a runbook for operating it, and evidence of the deployed analytics pipeline. The design prioritises zero-downtime deployment and instant rollback through the blue-green pattern, immutable per-commit image tagging, and fully automated TLS and Lambda provisioning — while the security and ethics review in Sections 4 and 5 identifies specific, concrete gaps (consent for tracking, email-sending abuse potential, seat-count race conditions, and secrets/supply-chain hardening) that would need to be closed before this design could be considered production-ready for handling real user data.